B.Tech CSE — Core Subjects Interview Q&A

Most Frequently Asked Questions by Technical Interviewers

Covers: OS · DBMS · CN · OOP · DS&A · Compiler Design · TOC · Software Engineering · Computer Organisation

SUBJECT 1 — OPERATING SYSTEMS (OS)

Q1. What is an Operating System? What are its main functions?

An OS is system software that manages hardware and software resources and provides services for programs. Main functions: Process Management (scheduling, creation, termination), Memory Management (allocation, paging, segmentation), File System Management, I/O Device Management, Security & Protection, and providing a User Interface.

Tip: GATE & interview staple — always mention the 5 core functions.

Q2. What is a process vs a thread? What is multi-threading?

Process: an independent program in execution with its own memory space (code, data, heap, stack). Thread: a lightweight unit of execution within a process; threads share the same memory space (code, data, heap) but have separate stacks and registers. Multi-threading: multiple threads within a process executing concurrently, improving responsiveness and CPU utilisation.

```
Process: own address space, heavy context switch
Thread: shared address space, lighter context switch
Types: User-level threads (managed by user library), Kernel-level threads (managed by OS)
```

Q3. Explain process states and the process state diagram.

A process moves through: New (being created) → Ready (waiting for CPU) → Running (currently executing) → Waiting/Blocked (waiting for I/O or event) → Terminated (finished). Transitions: Ready→Running (scheduled), Running→Waiting (I/O request), Waiting→Ready (I/O complete), Running→Ready (preempted), Running→Terminated (exit).

Tip: Draw this diagram in interviews — it shows clarity of thought.

Q4. What is a context switch? What information is saved/restored?

A context switch is saving the state of the current process and loading the state of the next process to run. Saved in PCB (Process Control Block): program counter, CPU registers, process state, memory management info, I/O status, accounting info. Context switches are pure overhead — OS does no useful work during them.

Q5. Explain CPU scheduling algorithms: FCFS, SJF, Round Robin, Priority, MLQS.

FCFS (First Come First Serve): non-preemptive, simple, suffers from convoy effect. SJF (Shortest Job First): minimum average waiting time, but requires knowing burst time in advance; can cause starvation. Round Robin: preemptive, each process gets a time quantum — fair but higher context switch overhead. Priority Scheduling: preemptive/non-preemptive; starvation solved by aging. MLQS (Multilevel Queue): separate queues per process type with fixed priority.

```
Metric: Average Waiting Time = (sum of waiting times) / n
AT=Arrival Time, BT=Burst Time, CT=Completion Time
TAT = CT - AT,  WT = TAT - BT
```

Q6. What is deadlock? Explain the 4 necessary conditions (Coffman conditions).

Deadlock: a set of processes each waiting for a resource held by another — circular wait with no progress. Coffman conditions (ALL must hold simultaneously): 1) Mutual Exclusion — resource can't be shared. 2) Hold and Wait — process holds resources while waiting for more. 3) No Preemption — resources can't be forcibly taken. 4) Circular Wait — circular chain of processes each waiting for next.

Tip: Prevention: break any ONE of the 4 conditions. Detection: Resource Allocation Graph. Recovery: preemption or rollback.

Q7. Explain deadlock avoidance — Banker's Algorithm.

Banker's Algorithm is a deadlock avoidance algorithm by Dijkstra. It checks if granting a resource request leads to a 'safe state' — a sequence in which every process can complete. Key matrices: Allocation (current), Max (maximum need), Need = Max - Allocation, Available. A state is safe if there exists a safe sequence; unsafe ≠ deadlock but may lead to one.

```
Safe sequence check:
1. Find process whose Need <= Available
2. If found, add its Allocation to Available (it finishes)
3. Repeat until all processes finish (safe) or no such process (unsafe)
```

Q8. What is memory management? Explain paging vs segmentation.

Paging: physical memory divided into fixed-size frames; logical memory into equal-size pages. OS maintains page table mapping page → frame. Eliminates external fragmentation but causes internal fragmentation. Segmentation: logical memory divided into variable-size segments (code, data, stack). More natural programming model. Causes external fragmentation. Modern systems use segmentation with paging (Intel x86).

```
Paging:      Logical addr = (page no, offset) → frame no via page table
Segmentation: Logical addr = (segment no, offset) → base + offset
```

Q9. What is virtual memory? Explain demand paging and page replacement algorithms.

Virtual memory allows processes to use more memory than physically available — pages loaded on demand from disk. Demand paging: load page only when accessed; a page fault triggers loading from disk. Page replacement needed when memory full. Algorithms: FIFO (oldest page out, Belady's anomaly), LRU (least recently used — optimal in practice), Optimal (OPT, replace page used farthest in future — theoretical best), Clock (approximation of LRU).

```
Page fault rate affects performance critically.
Thrashing: too many page faults, CPU spends more time swapping than executing.
Fix: working set model / increase RAM
```

Q10. What is the difference between internal and external fragmentation?

Internal fragmentation: wasted space INSIDE an allocated block (allocated more than requested). Occurs in fixed-size partition allocation and paging. External fragmentation: enough total free memory exists but it's non-contiguous — no single block large enough. Occurs in dynamic partitioning and segmentation. Solution: compaction (for external), best-fit allocation, paging.

Q11. Explain semaphores. What is the producer-consumer problem?

Semaphore is a synchronisation primitive — an integer variable with two atomic operations: wait(P): decrement; if negative, block. signal(V): increment; wake waiting process. Binary semaphore (mutex): 0 or 1 — used for mutual exclusion. Counting semaphore: tracks resource count. Producer-Consumer: producer adds to buffer; consumer removes. Use semaphores: empty (buffer slots), full (items), mutex (mutual exclusion).

```
semaphore mutex=1, empty=N, full=0;
Producer: wait(empty); wait(mutex); add item; signal(mutex); signal(full);
Consumer: wait(full);  wait(mutex); remove item; signal(mutex); signal(empty);
```

Q12. What is the difference between mutex and semaphore?

Mutex (Mutual Exclusion lock): ownership-based binary lock — only the thread that locked it can unlock it. Used for protecting critical sections. Semaphore: signalling mechanism — one thread can signal another. No ownership. Binary semaphore resembles mutex but lacks ownership semantics. Semaphores can have count > 1. Mutex is for mutual exclusion; semaphore is for synchronisation/signalling.

Q13. What are the differences between process synchronisation methods: mutex, semaphore, monitor, spinlock?

Spinlock: busy-wait loop checking lock — wastes CPU; good only for very short waits on multi-core. Mutex: blocking lock — OS suspends thread; good for longer waits. Semaphore: generalised counter with blocking. Monitor: high-level synchronisation construct with condition variables — built into languages (Java synchronized, Python threading.Condition). Monitors are easiest to use correctly.

Q14. What is disk scheduling? Explain FCFS, SSTF, SCAN, C-SCAN.

Disk scheduling minimises seek time (head movement). FCFS: service in order of arrival — fair but high seek time. SSTF (Shortest Seek Time First): serve nearest request — low seek time but starvation possible. SCAN (Elevator): head moves in one direction servicing requests, reverses at end. C-SCAN (Circular SCAN): moves in one direction only, jumps back to start — more uniform wait times.

Tip: SCAN is most used in practice. C-SCAN gives more uniform response times.

Q15. What are the types of operating systems?

Batch OS: jobs collected, processed in batches without user interaction. Time-Sharing OS: multiple users share CPU via rapid context switching (UNIX). Real-Time OS (RTOS): strict timing guarantees — hard RTOS (miss deadline = failure), soft RTOS (miss deadline = degraded). Distributed OS: manages cluster of computers as one system. Embedded OS: resource-constrained devices (FreeRTOS, VxWorks). Network OS: NFS, Novell NetWare.

SUBJECT 2 — DATABASE MANAGEMENT SYSTEMS (DBMS)

Q16. What is DBMS? Difference between DBMS and RDBMS?

DBMS: software to store, retrieve, and manage data (e.g., file system with added features). RDBMS: relational DBMS where data is stored in tables with relationships enforced via primary/foreign keys, supports SQL, ACID properties, and normalization. Examples: DBMS — IMS, dBASE. RDBMS — MySQL, PostgreSQL, Oracle, SQL Server.

Q17. What are ACID properties? Explain each.

ACID ensures reliable database transactions. Atomicity: transaction is all-or-nothing (commit or rollback). Consistency: DB moves from one valid state to another — constraints always satisfied. Isolation: concurrent transactions appear to execute sequentially — intermediate states invisible to others. Durability: committed transaction persists even after system failure (via WAL/redo logs).

Tip: All four must hold for a reliable transaction system. NoSQL databases often relax ACID for performance (BASE model).

Q18. Explain normalization. What are 1NF, 2NF, 3NF, and BCNF?

Normalization eliminates redundancy and dependency anomalies. 1NF: atomic values, no repeating groups, each column has unique name. 2NF: 1NF + no partial dependency (non-key attribute depends on whole primary key, not part). 3NF: 2NF + no transitive dependency (non-key attribute depends only on primary key, not on another non-key). BCNF: stricter 3NF — every determinant must be a candidate key.

```
Example: Student(RollNo, Name, CourseID, CourseName, InstructorID)
1NF: already atomic
2NF: CourseName depends on CourseID (partial dep) → separate table
3NF: InstructorID → CourseName (transitive) → further separate
```

Q19. What are primary key, foreign key, unique key, and candidate key?

Candidate key: minimal set of attributes uniquely identifying a tuple. Primary key: chosen candidate key — NOT NULL, UNIQUE, one per table. Foreign key: attribute(s) referencing primary key of another table — enforces referential integrity. Unique key: ensures uniqueness but can have NULL (unlike PK). Super key: any set of attributes uniquely identifying a row (superset of candidate key).

Q20. What is the difference between WHERE and HAVING clauses?

WHERE: filters rows BEFORE grouping — cannot use aggregate functions. HAVING: filters groups AFTER GROUP BY — can use aggregate functions. WHERE applies to individual rows; HAVING applies to grouped results.

```
SELECT dept, COUNT(*) as cnt
FROM employees
WHERE salary > 30000        -- filter rows first
GROUP BY dept
HAVING COUNT(*) > 5;         -- filter groups
```

Q21. Explain JOINs: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF.

INNER JOIN: rows matching in both tables. LEFT JOIN: all rows from left + matched from right (NULL if no match). RIGHT JOIN: all rows from right + matched from left. FULL OUTER JOIN: all rows from both tables (NULLs where no match). CROSS JOIN: cartesian product (every combination). SELF JOIN: table joined with itself (e.g., employee-manager hierarchy).

```
SELECT e.name, m.name as manager
FROM employees e
INNER JOIN employees m ON e.manager_id = m.id;  -- SELF JOIN
```

Q22. What is an index in DBMS? Types of indexes?

Index is a data structure (B+ Tree or Hash) on table columns to speed up retrieval. Types: Primary index (on ordered key field), Secondary index (on non-ordering field), Clustered index (data rows physically sorted by index key — one per table), Non-clustered index (separate structure with pointers to rows — multiple per table), Composite index (multiple columns), Covering index (all query columns in index).

Tip: Too many indexes slow down INSERT/UPDATE/DELETE. Index the columns in WHERE, JOIN, ORDER BY clauses.

Q23. What is a transaction? What are different transaction states?

Transaction: a logical unit of work — set of operations executed atomically. States: Active (executing) → Partially Committed (last statement executed) → Committed (changes permanent) → Failed (error detected) → Aborted (rolled back, DB restored). A transaction moves from Partially Committed to Committed only after writing to disk.

Q24. Explain concurrency control: locking protocols, two-phase locking (2PL).

2PL ensures serializability. Two phases: Growing phase — acquire locks, no release. Shrinking phase — release locks, no acquire. Variants: Basic 2PL (may deadlock), Strict 2PL (release all locks at commit — prevents cascading rollback), Rigorous 2PL (hold all locks till commit). Lock types: Shared (S) for read, Exclusive (X) for write.

Tip: 2PL guarantees conflict serializability but can cause deadlocks.

Q25. What is SQL injection? How do you prevent it?

SQL injection: attacker inserts malicious SQL into input that gets executed by the DB. E.g., username = "' OR '1'='1" bypasses authentication. Prevention: 1) Parameterized queries/prepared statements, 2) Stored procedures, 3) Input validation/sanitization, 4) ORMs (Prisma, Hibernate), 5) Principle of least privilege for DB accounts.

```
-- Vulnerable:
SELECT * FROM users WHERE name = '" + userInput + "';

-- Safe (prepared statement):
SELECT * FROM users WHERE name = ?;  -- bind parameter separately
```

Q26. What is the difference between DELETE, TRUNCATE, and DROP?

DELETE: DML — removes specific rows (WHERE clause); logs each row deletion, can rollback, fires triggers. TRUNCATE: DDL — removes ALL rows; faster (minimal logging), cannot rollback in most DBs, resets identity, doesn't fire row triggers. DROP: DDL — removes entire table structure + data + indexes + constraints permanently; cannot rollback.

Q27. Explain ER model — entities, attributes, relationships, cardinality.

Entity: real-world object (Student, Course). Attribute: property of entity (name, id). Relationship: association between entities (Student ENROLLS IN Course). Cardinality: 1:1 (person-passport), 1:N (customer-orders), M:N (student-courses). Strong entity: has its own key. Weak entity: depends on strong entity (Order line depends on Order). Participation: total (every entity participates) or partial.

Q28. What is denormalization? When would you use it?

Denormalization intentionally introduces redundancy into a normalized schema to improve read performance by reducing JOIN operations. Used in: read-heavy systems, data warehouses, reporting dashboards, OLAP systems, caching layers. Tradeoff: faster reads vs. slower writes and risk of data inconsistency. Modern approach: normalize the source DB, denormalize in read replicas or data warehouses.

SUBJECT 3 — COMPUTER NETWORKS (CN)

Q29. Explain the OSI model — all 7 layers with functions.

Physical (L1): bit transmission over medium (cables, signals, hubs). Data Link (L2): framing, MAC addressing, error detection (Ethernet, switches). Network (L3): logical addressing, routing (IP, routers). Transport (L4): end-to-end communication, reliability (TCP/UDP). Session (L5): session management. Presentation (L6): data formatting, encryption, compression. Application (L7): user-facing protocols (HTTP, FTP, DNS, SMTP).

Tip: Mnemonic: 'Please Do Not Throw Sausage Pizza Away' (Physical→Application) or reverse.

Q30. Compare TCP vs UDP. When would you use each?

TCP (Transmission Control Protocol): connection-oriented, reliable (acknowledgements, retransmission), ordered delivery, flow control, congestion control, slower. UDP (User Datagram Protocol): connectionless, unreliable (no ACK), no ordering, no flow control, faster, lower overhead. Use TCP for: web (HTTP), email (SMTP), file transfer (FTP). Use UDP for: video streaming, gaming, DNS, VoIP, DHCP.

```
TCP: 3-way handshake (SYN → SYN-ACK → ACK) before data
UDP: no handshake, fire-and-forget datagrams
```

Q31. What is the TCP 3-way handshake? What is the 4-way termination?

3-way handshake (connection establishment): 1) Client sends SYN (seq=x). 2) Server responds SYN-ACK (seq=y, ack=x+1). 3) Client sends ACK (ack=y+1). Connection established. 4-way termination: 1) Client FIN. 2) Server ACK. 3) Server FIN. 4) Client ACK → wait TIME_WAIT (2*MSL) before fully closing.

Q32. What is an IP address? Difference between IPv4 and IPv6? Explain subnetting.

IP address: unique logical address assigned to each device. IPv4: 32-bit, dotted decimal (192.168.1.1), ~4.3B addresses — exhausted. IPv6: 128-bit, hex colon notation (2001:db8::1), ~340 undecillion addresses, built-in IPSec. Subnetting: dividing a network into smaller subnetworks using subnet mask. CIDR notation: 192.168.1.0/24 means 24 bits for network, 8 bits for host (254 usable hosts).

```
Subnet /24: 256 addresses, 254 usable (network + broadcast reserved)
Subnet /28: 16 addresses, 14 usable
Sub mask /24 = 255.255.255.0
```

Q33. What is DNS? How does DNS resolution work?

DNS (Domain Name System) translates human-readable domain names to IP addresses. Resolution: 1) Browser checks local cache. 2) Query OS resolver (hosts file, local cache). 3) Query Recursive Resolver (ISP). 4) Resolver queries Root nameserver (.com, .org). 5) Root points to TLD nameserver (.com). 6) TLD points to Authoritative nameserver (example.com). 7) Authoritative returns IP. TTL determines cache duration.

Q34. What is HTTP vs HTTPS? What is TLS/SSL?

HTTP: HyperText Transfer Protocol — application layer protocol for web communication, stateless, plain text. HTTPS: HTTP over TLS — encrypted, authenticated, integrity-protected. TLS (Transport Layer Security):

cryptographic protocol using asymmetric encryption (for key exchange) + symmetric encryption (for data). SSL is deprecated predecessor of TLS. HTTPS uses port 443; HTTP uses port 80.

```
TLS handshake: Client Hello → Server Hello + Certificate → Key Exchange →
Change Cipher Spec → Encrypted communication begins
```

Q35. What are HTTP methods? Explain GET, POST, PUT, PATCH, DELETE.

GET: retrieve resource, idempotent, cacheable, params in URL. POST: create resource, non-idempotent, body payload. PUT: replace entire resource, idempotent. PATCH: partial update, non-idempotent. DELETE: remove resource, idempotent. HEAD: like GET but no body (check metadata). OPTIONS: get supported methods (used in CORS preflight).

Q36. What is the difference between a hub, switch, and router?

Hub (L1): broadcasts data to ALL ports — collisions, inefficient. Switch (L2): learns MAC addresses, forwards frames only to destination port — no collisions per port. Router (L3): connects different networks, routes packets based on IP addresses using routing tables. Modern networks: access switches, distribution switches, routers at network edge, firewalls.

Q37. What is NAT (Network Address Translation)?

NAT translates private IP addresses (192.168.x.x, 10.x.x.x) to a single public IP for internet access — solving IPv4 exhaustion. Types: Static NAT (1:1 mapping), Dynamic NAT (pool of public IPs), PAT/Masquerading (many:1, uses port numbers to track connections — most common in home routers). NAT breaks end-to-end connectivity but provides a layer of security.

Q38. Explain congestion control in TCP: slow start, congestion avoidance, fast retransmit.

TCP congestion control prevents overwhelming the network. Slow Start: begin with cwnd=1 MSS, double every RTT until ssthresh. Congestion Avoidance: increase cwnd by 1 per RTT (linear). Fast Retransmit: on 3 duplicate ACKs, retransmit lost segment without waiting for timeout. Fast Recovery: set ssthresh=cwnd/2, cwnd=ssthresh+3, continue congestion avoidance (Reno). Cubic/BBR are modern algorithms.

Q39. What are common network protocols and their port numbers?

HTTP:80, HTTPS:443, FTP:20(data)/21(control), SSH:22, Telnet:23, SMTP:25, DNS:53, DHCP:67/68, POP3:110, IMAP:143, SNMP:161, RDP:3389, MySQL:3306, PostgreSQL:5432, MongoDB:27017, Redis:6379.

Tip: Port 0-1023: well-known ports. 1024-49151: registered. 49152-65535: dynamic/ephemeral.

Q40. What is a firewall? Types and how they work?

Firewall: network security device that monitors and controls incoming/outgoing traffic based on rules. Types: Packet Filter (L3/L4 — checks IP/port, stateless, fast). Stateful Firewall (tracks connection state — more secure). Application-layer/Proxy Firewall (deep packet inspection at L7). Next-Gen Firewall (NGFW): combines all above + IPS, DPI, application awareness. WAF (Web Application Firewall): protects web apps from XSS, SQLi, etc.

SUBJECT 4 — OBJECT-ORIENTED PROGRAMMING (OOP)

Q41. What are the 4 pillars of OOP? Explain each.

Encapsulation: bundling data + methods that operate on it; hiding internal state (private fields, public methods). Abstraction: exposing only necessary interface, hiding implementation details (abstract classes, interfaces). Inheritance: a class acquires properties/methods of another class — promotes code reuse. Polymorphism: same interface, different implementations — method overloading (compile-time) & overriding (runtime).

Q42. What is the difference between method overloading and method overriding?

Overloading (Compile-time/Static polymorphism): same method name, different parameters (type/number) in the SAME class. Resolved at compile time. Overriding (Runtime/Dynamic polymorphism): subclass provides different implementation of parent class method with same signature. Resolved at runtime via vtable. In Java, @Override annotation; in C++, virtual keyword.

```
// Overloading
void print(int x) {}
void print(String s) {}

// Overriding
class Animal { virtual void sound() { cout << 'generic'; } };
class Dog : Animal { void sound() override { cout << 'Woof'; } };
```

Q43. What is an abstract class vs an interface?

Abstract class: may have abstract (unimplemented) and concrete methods; can have instance variables; a class can extend only ONE abstract class (single inheritance in Java). Interface: all methods abstract (pre-Java 8); no instance variables (only constants); a class can implement MULTIPLE interfaces. Use abstract class for 'is-a' with shared code; interface for 'can-do' capability contracts.

Tip: Java 8+ interfaces can have default and static methods. Java 9+ adds private methods.

Q44. What is inheritance? Types of inheritance?

Inheritance: child class inherits properties and methods of parent class, promoting code reuse and hierarchy. Types: Single (A→B), Multilevel (A→B→C), Hierarchical (A→B, A→C), Multiple (B,C→D — supported in C++ via virtual base class, simulated in Java via interfaces), Hybrid (combination). Java doesn't support multiple class inheritance to avoid the Diamond Problem.

Q45. What is the Diamond Problem? How is it solved?

Diamond Problem: in multiple inheritance, if class D inherits from B and C which both inherit from A, and B/C both override a method of A, D has ambiguity about which version to use. Solution in C++: virtual base class (single shared copy of A). Solution in Java: interfaces (default method conflict resolved by overriding in D). Python: MRO (Method Resolution Order) — C3 linearization.

Q46. What is polymorphism? Explain runtime polymorphism with vtable.

Polymorphism: 'many forms' — same interface, different behavior. Runtime polymorphism via virtual functions uses a vtable (virtual function table): each class with virtual functions has a vtable of function pointers. When calling a virtual method on a pointer/reference, the vtable is looked up at runtime to call the correct overridden version.

```
Animal* a = new Dog();
a->sound(); // calls Dog::sound() via vtable, not Animal::sound()
```

Q47. What are constructors and destructors? Types of constructors?

Constructor: special method called at object creation — same name as class, no return type. Initializes object state. Destructor: called at object destruction — releases resources. Types of constructors: Default (no args), Parameterized (with args), Copy constructor (takes same-type reference, creates copy), Move constructor (C++11, transfers resources from temp object). In Java, no destructors — GC + finalize().

Q48. What are access specifiers: public, private, protected?

public: accessible everywhere. private: accessible only within the same class. protected: accessible within the class and its subclasses. Default/package-private (Java): accessible within same package. In C++, class members are private by default; struct members are public by default. Inheritance visibility also affects access in subclasses.

Q49. What is the 'this' keyword in OOP?

'this' is a reference to the current object instance within a method or constructor. Uses: 1) Differentiate instance variables from local variables with same name. 2) Call another constructor (this()) — constructor chaining. 3) Pass current object as argument. 4) Return current object from methods (method chaining). In static methods, 'this' is not available.

Q50. What are design patterns? Explain Singleton, Factory, Observer.

Design patterns are reusable solutions to common software design problems. Singleton: ensures only one instance exists globally (e.g., DB connection pool, Logger). Factory: creates objects without specifying exact class — caller uses factory method. Observer: object (Subject) maintains list of dependents (Observers) notified on state change — e.g., event listeners, MVC.

```
// Singleton (thread-safe)
public class DB {
  private static DB instance;
  private DB() {}
  public static synchronized DB getInstance() {
    if (instance == null) instance = new DB();
    return instance;
  }
}
```

Q51. What is the difference between composition and inheritance? Which is preferred?

Inheritance ('is-a'): class extends another, inheriting all properties — tight coupling, breaks encapsulation. Composition ('has-a'): class contains instance of another class as member — loose coupling, more flexible. Prefer composition over inheritance (GoF principle) — it allows changing behaviour at runtime, avoids fragile base class problem, and supports better unit testing.

SUBJECT 5 — DATA STRUCTURES & ALGORITHMS (DS&A;)

Q52. What is Big O notation? Give the time complexity of common operations.

Big O describes the upper bound of an algorithm's growth rate as input size n increases. Common complexities: O(1) — constant (hash map access). O(log n) — binary search. O(n) — linear scan. O(n log n) — merge sort. O(n²) — bubble sort. O(2^n) — subset generation. O(n!) — permutations. Always analyse worst case unless specified.

```
Data structure operations:
Array: access O(1), search O(n), insert/delete O(n)
LinkedList: access O(n), insert/delete head O(1)
HashMap: get/put O(1) avg, O(n) worst
BST: search O(log n) balanced, O(n) skewed
```

Q53. Compare arrays vs linked lists. When to use each?

Array: contiguous memory, O(1) random access, cache-friendly, fixed size (or dynamic with amortised O(1) append), O(n) insert/delete middle. Linked List: non-contiguous nodes with pointers, O(n) access, O(1) insert/delete at known node, no size limit. Use array when random access needed; linked list when frequent insertions/deletions at ends or middle.

Q54. Explain stack and queue. What are their applications?

Stack (LIFO — Last In First Out): push/pop at top. O(1) operations. Applications: function call stack, undo/redo, expression evaluation, DFS, balanced parentheses check. Queue (FIFO — First In First Out): enqueue at rear, dequeue from front. O(1) operations. Applications: BFS, printer queue, CPU scheduling, message queues. Deque: both ends. Priority Queue: min/max heap.

Q55. What is a Binary Search Tree (BST)? Explain insert, delete, search.

BST: binary tree where left child < parent < right child. Search: compare and go left/right — O(log n) balanced, O(n) skewed. Insert: find correct position maintaining BST property — O(log n). Delete: 3 cases — leaf (remove), one child (replace with child), two children (replace with inorder successor/predecessor then delete that node).

```
Inorder traversal of BST gives sorted output — a key property!
Balanced BST (AVL, Red-Black): guarantees O(log n) all ops
```

Q56. What is a heap? How is heap sort implemented?

Heap: complete binary tree where each node satisfies heap property. Min-heap: parent ≤ children (root = minimum). Max-heap: parent ≥ children (root = maximum). Implemented as array: parent at i, left child at 2i+1, right at 2i+2. Heap sort: 1) Build max-heap from array O(n). 2) Repeatedly extract max (swap root with last, heapify) O(n log n). Total: O(n log n), in-place, not stable.

Q57. What is dynamic programming? Explain with an example.

DP: solving problems by breaking into overlapping subproblems, storing results (memoization/tabulation) to avoid recomputation. Key properties: optimal substructure (global optimum from subproblem optima) + overlapping subproblems. Example: Fibonacci — naive O(2^n), DP O(n). Classic problems: Longest Common Subsequence, 0/1 Knapsack, Shortest Path (Bellman-Ford), Matrix Chain Multiplication.

```
// Fibonacci DP (bottom-up tabulation)
int fib(int n) {
  int dp[n+1]; dp[0]=0; dp[1]=1;
  for(int i=2;i<=n;i++) dp[i]=dp[i-1]+dp[i-2];
  return dp[n];
} // O(n) time, O(n) space
```

Q58. Compare sorting algorithms: Bubble, Selection, Insertion, Merge, Quick, Heap.

Bubble Sort: O(n²) avg/worst, O(1) space, stable. Selection Sort: O(n²), O(1), not stable. Insertion Sort: O(n²) worst, O(n) best (nearly sorted), O(1), stable — best for small/nearly sorted. Merge Sort: O(n log n), O(n) space, stable — preferred for linked lists. Quick Sort: O(n log n) avg, O(n²) worst (bad pivot), O(log n) space, not stable — fastest in practice. Heap Sort: O(n log n), O(1), not stable.

Q59. What is a hash table? How is collision handled?

Hash table: maps keys to values using hash function h(key) → index. O(1) avg for insert/search/delete. Hash function properties: deterministic, uniform distribution, fast computation. Collision (two keys → same index) handling: Chaining — linked list at each bucket (O(n) worst). Open Addressing — probe next slots: Linear Probing, Quadratic Probing, Double Hashing. Load factor = n/capacity; rehash when load factor exceeds threshold (0.7).

Q60. Explain graph representations and BFS vs DFS.

Graph representations: Adjacency Matrix — O(V²) space, O(1) edge check. Adjacency List — O(V+E) space, better for sparse graphs. BFS (Breadth-First Search): level-by-level using queue — O(V+E). Applications: shortest path (unweighted), level order traversal, peer-to-peer networks. DFS (Depth-First Search): explore as deep as possible using stack/recursion — O(V+E). Applications: topological sort, cycle detection, connected components, maze solving.

```
BFS: while queue not empty: dequeue v, process, enqueue unvisited neighbours
DFS: for each unvisited neighbour of v: mark visited, recurse(neighbour)
```

Q61. What is a trie? What are its applications?

Trie (Prefix Tree): tree where each node represents a character; root to node path represents a prefix. O(L) insert/search/delete where L = key length — independent of number of keys. Applications: autocomplete, spell checker, IP routing (longest prefix match), dictionary words, phone book, word games (Boggle). Space-efficient alternative: Compressed Trie (Patricia Trie), Ternary Search Tree.

Q62. Explain Dijkstra's and Bellman-Ford algorithms. Key differences?

Dijkstra: single-source shortest path, greedy approach using min-heap, O((V+E) log V). Works only for non-negative edge weights. Bellman-Ford: O(VE) — slower but handles negative weights; detects negative cycles. Use Dijkstra for road maps, GPS; Bellman-Ford for currency arbitrage detection, networks with negative edges. Floyd-Warshall: all-pairs shortest path $O(V^3)$.

SUBJECT 6 — COMPILER DESIGN

Q63. What are the phases of a compiler?

Lexical Analysis (Scanner): source code → tokens (identifiers, keywords, literals). Syntax Analysis (Parser): tokens → parse tree/AST using grammar rules (CFG). Semantic Analysis: type checking, scope resolution, semantic errors. Intermediate Code Generation: AST → three-address code or IR. Code Optimisation: improve IR (constant folding, dead code elimination, loop optimisation). Code Generation: IR → target machine code. Symbol table and error handler cross-cut all phases.

Tip: Frontend: lexical + syntax + semantic. Backend: IR generation + optimisation + code gen.

Q64. What is a lexer (lexical analyser)? What are tokens, lexemes, patterns?

Lexer reads source characters and groups them into tokens. Token: category (IDENTIFIER, NUMBER, KEYWORD, OPERATOR). Lexeme: actual string matched ('x', '42', 'if', '+'). Pattern: rule (regex) defining what strings match a token. Built with: Regular Expressions → NFA (Thompson's construction) → DFA (subset construction) → minimised DFA. Tools: Lex/Flex.

Q65. What is a Context-Free Grammar (CFG)? What are parse trees and ambiguity?

CFG: formal grammar G = (V, T, P, S) where V=variables, T=terminals, P=productions, S=start symbol. Derivation: replace variables using production rules. Parse tree: tree representation of derivation. Ambiguous grammar: a string has more than one parse tree (two different leftmost derivations). Ambiguity is a problem for compilers — resolved by grammar rewriting or precedence/associativity rules.

Q66. What is the difference between top-down and bottom-up parsing?

Top-Down: starts from root (start symbol), expands productions to match input. LL parsers — predictive parsing with lookahead. LL(1): 1 lookahead token, uses parsing table, non-recursive (table-driven) or recursive descent. Bottom-Up: starts from leaves (input), reduces using productions until start symbol. LR parsers (LR(0), SLR, LALR, CLR) — more powerful than LL. LALR(1) used in practice (Yacc/Bison).

Q67. What is a symbol table? What information does it store?

Symbol table: data structure maintained throughout compilation storing information about identifiers. Stores: name, type, scope, memory location (offset), function signature (parameter types, return type), line of declaration. Implemented as hash table for O(1) lookup. Nested scopes implemented as stack of hash tables. Used in semantic analysis, code generation, and debugging (DWARF format).

Q68. What are common compiler optimisations?

Machine-independent: Constant Folding (2+3 → 5 at compile time), Constant Propagation, Dead Code Elimination (unreachable code), Common Subexpression Elimination (reuse computed values), Loop Invariant Code Motion (move loop-invariant computations out), Strength Reduction (replace x^2 with x*x, multiply by 2 with left shift). Machine-dependent: Register Allocation, Instruction Scheduling, Peephole Optimisation.

SUBJECT 7 — THEORY OF COMPUTATION (TOC)

Q69. What is a Finite Automaton (FA)? DFA vs NFA.

FA: mathematical model of computation — (Q, Σ, δ, q0, F) where Q=states, Σ=alphabet, δ=transition function, q0=start, F=accept states. DFA (Deterministic FA): exactly one transition per (state, input) — no epsilon transitions. NFA (Non-deterministic FA): zero or more transitions per (state, input), epsilon transitions allowed. Both recognise the same class of languages (Regular Languages). NFA→DFA via subset construction (may exponentially blow up states).

Q70. What are regular expressions? Give examples of operations.

Regular expressions describe regular languages. Operations: Union (a|b — match a or b), Concatenation (ab — match a then b), Kleene Star (a* — zero or more a's), Plus (a+ — one or more), Question Mark (a? — zero or one). Examples: [0-9]+ matches integers; [a-zA-Z_][a-zA-Z0-9_]* matches identifiers; (0|1)* matches binary strings. Regular expressions ↔ DFA/NFA ↔ Regular Grammars (Type 3 in Chomsky hierarchy).

Q71. What is the Pumping Lemma for regular languages?

Pumping Lemma: used to PROVE a language is NOT regular. If L is regular, then there exists pumping length p such that any string s ∈ L with |s| ≥ p can be split into xyz where: |y| ≥ 1, |xy| ≤ p, and xy^i z ∈ L for all i ≥ 0. To prove L is not regular: assume L is regular, find a string that cannot be pumped, contradiction.

```
Classic example: L = {a■b■ | n≥0} is NOT regular
Proof: take s = a■b■, any pump of y in aaa... portion
br breaks equal count balance — contradiction.
```

Q72. What is a Pushdown Automaton (PDA)? What languages do they recognise?

PDA: FA with an additional stack for memory. Recognises Context-Free Languages (CFLs) — strictly more powerful than FA. Configuration: (state, remaining input, stack contents). Transition: read input, pop/push stack symbol, change state. Two acceptance modes: empty stack or final state. Examples: balanced parentheses, palindromes, a■b■ — all CFL but not regular.

Q73. What is the Chomsky hierarchy of languages?

Type 0 (Recursively Enumerable): recognised by Turing Machine — most general. Type 1 (Context-Sensitive): recognised by Linear Bounded Automaton. Type 2 (Context-Free): recognised by Pushdown Automaton — used for programming language grammars. Type 3 (Regular): recognised by Finite Automaton — most restrictive. Each type is a proper subset of the one above.

Q74. What is a Turing Machine? What is the Church-Turing thesis?

Turing Machine (TM): theoretical model of computation — infinite tape, read/write head, states, transition function. Can simulate any algorithm. The Universal Turing Machine can simulate any other TM. Church-Turing Thesis: any function that can be 'effectively computed' can be computed by a Turing Machine — defines the limits of computation. Variants (multi-tape, non-deterministic) are all equivalent in power.

Q75. What is the Halting Problem? Why is it undecidable?

Halting Problem: given a TM M and input w, does M halt on w? Alan Turing proved (1936) this is undecidable — no algorithm can solve it for all inputs. Proof by contradiction (diagonalization): assume H solves halting problem; construct D that calls H and does the opposite; run D on itself — contradiction. Implications: many problems are undecidable; program verification is fundamentally limited.

SUBJECT 8 — SOFTWARE ENGINEERING (SE)

Q76. What are SDLC models? Compare Waterfall, Agile, and Scrum.

SDLC phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance. Waterfall: sequential, rigid, each phase complete before next — good for well-defined requirements. Agile: iterative, incremental, adaptive to change — 12 principles, frequent delivery. Scrum: Agile framework with Sprints (2-4 week iterations), Product Backlog, Sprint Backlog, Daily Standup, Sprint Review, Retrospective. Kanban: continuous

flow, WIP limits.

Q77. What is software testing? Types of testing?

Unit Testing: test individual functions/modules in isolation (Jest, JUnit). Integration Testing: test module interactions. System Testing: test complete system. Acceptance Testing: verify against user requirements (UAT). Regression Testing: ensure changes don't break existing functionality. Performance Testing: load, stress, scalability. White Box: knows internal code. Black Box: tests only inputs/outputs. Grey Box: partial knowledge.

Q78. What are SOLID principles?

S — Single Responsibility: class should have only one reason to change. O — Open/Closed: open for extension, closed for modification (add new code, don't change existing). L — Liskov Substitution: subclass objects should be substitutable for parent class objects. I — Interface Segregation: many specific interfaces better than one general-purpose. D — Dependency Inversion: depend on abstractions, not concretions (use interfaces).

Tip: SOLID principles are the most asked OOP design principles in senior-level interviews.

Q79. What is software coupling and cohesion?

Coupling: degree of interdependence between modules — low coupling is desired (changes in one module minimally affect others). Types: Content > Common > External > Control > Stamp > Data (weakest, best). Cohesion: degree to which elements of a module belong together — high cohesion is desired (module does one thing well). Types: Coincidental < Logical < Temporal < Procedural < Communicational < Sequential < Functional (best).

Q80. What is version control? Explain Git workflow.

Version control tracks changes to code over time, enables collaboration. Git: distributed VCS. Key commands: git init/clone, add, commit, push, pull, branch, merge, rebase, stash. Git Flow: main (production), develop, feature/*, release/*, hotfix/*. GitHub Flow: simpler — main + feature branches + pull requests. Conflict resolution: merge commits or rebasing.

```
git checkout -b feature/login   # create feature branch
git add . && git commit -m 'feat: add login'
git push origin feature/login
# Create PR → Code Review → Merge to main
```

Q81. What is the difference between functional and non-functional requirements?

Functional requirements: WHAT the system should do — specific behaviors, functions, inputs/outputs. Examples: 'User shall be able to login with email/password', 'System shall generate monthly reports'. Non-functional requirements: HOW the system should perform — quality attributes. Examples: Performance (response time < 200ms), Scalability (10,000 concurrent users), Security, Reliability (99.9% uptime), Maintainability, Portability.

SUBJECT 9 — COMPUTER ORGANISATION & ARCHITECTURE (CO&A;)

Q82. What is the difference between RISC and CISC?

RISC (Reduced Instruction Set Computer): few simple fixed-length instructions, load/store architecture, many registers, pipelining-friendly, simple decoder. Examples: ARM, MIPS, RISC-V. CISC (Complex Instruction Set Computer): many complex variable-length instructions, can operate directly on memory, fewer registers, complex decoder. Examples: x86, x86-64. Modern CPUs decode CISC externally to RISC-like micro-ops internally.

Q83. What is pipelining? What are pipeline hazards?

Pipelining: overlap execution of multiple instructions — each stage processes a different instruction simultaneously. 5-stage: IF (Instruction Fetch), ID (Decode), EX (Execute), MEM (Memory), WB (Write Back). Hazards: Data Hazard (instruction needs result not yet computed — forwarding/stalling). Control Hazard (branch outcome unknown — branch prediction, flushing). Structural Hazard (two instructions need same resource — stall

or duplicate resource).

Q84. Explain cache memory. What is cache hierarchy?

Cache: fast SRAM between CPU and main memory reducing average memory access time. Cache hierarchy: L1 (fastest, smallest, per core, ~32KB, ~1-4 cycles), L2 (medium, per core, ~256KB, ~10 cycles), L3 (shared, ~8-32MB, ~30 cycles), RAM (~100 cycles), Disk (~millions). Cache performance: Hit rate, Miss rate, Hit time, Miss penalty. Policies: Write-through vs Write-back; Direct-mapped vs Set-associative vs Fully-associative.

Q85. What is the difference between synchronous and asynchronous communication?

Synchronous: sender waits (blocks) until receiver acknowledges — tightly coupled, simpler to program. Asynchronous: sender does not wait — decoupled, higher throughput but more complex (callbacks, message queues, polling). Examples: Synchronous — function call, HTTP request-response. Asynchronous — message queues (Kafka, RabbitMQ), WebSockets, event-driven programming, async/await in JavaScript.

Q86. What are number systems used in computing? Explain 2's complement.

Binary (base 2), Octal (base 8), Decimal (base 10), Hexadecimal (base 16). 2's Complement: standard representation of signed integers. Positive numbers: normal binary. Negative numbers: invert all bits (1's complement) + 1. Benefits: single representation of zero, arithmetic works uniformly, subtraction = addition of negative. Range for n bits: -2^(n-1) to 2^(n-1) - 1. Example: -5 in 8-bit: 11111011.

```
8-bit 2's complement of 5 (00000101):
Invert: 11111010
Add 1: 11111011 = -5 ✓
Verify: 11111011 + 00000101 = 100000000 (overflow, ignore carry) = 0 ✓
```

Total Questions	Subjects Covered	Candidate	Prepared For
86	9 Core Subjects	Nayan Mishra	B.Tech CSE Interviews